GMI German-Malaysian Institute

DIPLOMA PROGRAMME

PRACTICAL TEST - ANSWER

Academic Period : January 2022

Subject	:	**NETWORK PROGRAMMING**
Code	:	**SPG 0463**
Duration	:	**2 HOUR**
Total Marks	:	**30**

GENERAL INSTRUCTIONS:

1. Write down clearly (with capital letters) your name, ID no., semester, trade, group and the date of examination at the bottom of this page.
2. Do not open this question booklet until you are told to do so by the invigilator.
3. Read the specific instructions and questions carefully before answering.

This question paper consists of **3** printed pages including this page.

Student Name & ID	:	Abdul Hariz b Mohd Sarnudin (CBS24070208) Mohammad Danial b Norhisam (CBS24070222)
Semester/Trade/Group	:	Semester 3 / DCBS / CBS1
Date	:	3rd of November 2025

Table of Contents

Table of Contents **2**
Part 1 3
1. Code 3
Output 3
Part 2 4
2. a) 4
Output 4
2. b) 5
Output 5
Part 3 6
3. Dictionaries 6

Answer all questions.

Please share screenshot of your code & output as responses clearly. Submit answers in a separate PDF and number your responses (e.g. 1,2a,2c).

Part 1

1. Code

```
sent = "Do not worry about failure, worry about the CHANCES you miss when you do not even try"

#replace “worry” with “happy”
a = sent.replace( __old: "worry",  __new: "happy")
print("a.", a)

#find index of “failure” in the str
b = sent.find("failure")
print("b. The index is:", b)

#calculate the length of str
c = len(sent)
print("c. Length of string:", c)

#extract (pinpoint and print) “CHANCES” in str
d = sent[sent.find("CHANCES"):sent.find("CHANCES") + len("CHANCES")]
print("d.", d)

#capitalize the str
e = sent.capitalize()
print("e.", e)
```

Output

```
a. Do not happy about failure, happy about the CHANCES you miss when you do not even try
b. The index is: 19
c. Length of string: 85
d. CHANCES
e. Do not worry about failure, worry about the chances you miss when you do not even try

Process finished with exit code 0
```

Part 2

2. a)

```
def custom_unique_word_sorter_vowels(text_string):

    words = text_string.split()
    unique_words = list(set(words))

    #Define vowels
    VOWELS = 'aeiouAEIOU'

    #Sorting function
    def sort_key(word):

        is_consonant_start = word[0] not in VOWELS
        return (is_consonant_start, word)

    unique_words.sort(key=sort_key)

    return unique_words

#Define input and output (tk setup user input)
input_str_a = "apple orange banana apple egg igloo banana"
output_list_a = custom_unique_word_sorter_vowels(input_str_a)

#Print defined input and output
print(f"Input: {input_str_a}") #Variable dkt line 20
print(f"Output (Vowels First): {output_list_a}") #Variable dkt line 12
```

Output

```
"C:\Users\Abdul Hariz\AppData\Local\Microsoft\WindowsApps\python3.11.exe"
Input: apple orange banana apple egg igloo banana
Output (Vowels First): ['apple', 'egg', 'igloo', 'orange', 'banana']

Process finished with exit code 0
```

2. b)

```python
def ctf_word_organizer_basic(text_string):

    #Split the string into a list of words
    all_words = text_string.split()

    # We use a helper list and a set to manually track uniqueness and maintain order.
    unique_ordered_words = []
    seen_words = set()

    for word in all_words:
        # Check if the word starts with the high-priority prefix
        if word not in seen_words:
            unique_ordered_words.append(word)
            seen_words.add(word)

    #Separate the unique words into two groups
    key_words = []
    other_words = []
    KEY_PREFIX = "key_"

    for word in unique_ordered_words:
        if word.startswith(KEY_PREFIX):
            key_words.append(word)
        else:
            other_words.append(word)

    #Sort the 'key_' words alphabetically (default sort order)
    key_words.sort()

    #Sort all other words in reverse alphabetical order
    other_words.sort(reverse=True)

    #Define final variable with combine the two sorted lists
    final_sorted_list = key_words + other_words
    return final_sorted_list

#Define input (tk setup user input)
input_str_b = "noise1 key_alpha key_beta spg0453 python noise2 key_alpha noise1 noise7
noise10 hariz sir_ameer zap key_omega key_sigma"
output_list_b = ctf_word_organizer_basic(input_str_b)

print(f"Input: {input_str_b}")
print(f"Output (CTF Organized - Basic): {output_list_b}")
```

Output

```
"C:\Users\Abdul Hariz\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "C:\Users\Abdul Hariz\Documents\All Subject Notes\SEMESTER 3\CIT 0463 - Python\Code Test\Hemoglobin
Input: noise1 key_alpha key_beta spg0453 python noise2 key_alpha noise1 noise7 noise10 hariz sir_ameer zap key_omega key_sigma
Output (CTF Organized - Basic): ['key_alpha', 'key_beta', 'key_omega', 'key_sigma', 'zap', 'spg0453', 'sir_ameer', 'python', 'noise7', 'noise2', 'noise10', 'noise1', 'hariz']

Process finished with exit code 0
```

Part 3

3. Dictionaries

```
#the state name on the left is the 'key' and the abbreviation on the right is the 'value' of the dictionary
states = {
    "Oregon": "OR", #the key is 'Oregon', value is 'OR'
    "Florida": "FL", #the key is 'Florida', value is 'FL'
    "California": "CA", #the key is 'California', value is 'CA'
    "New York": "NY", #the key is 'New York', value is 'NY'
    "Michigan": "MI" #the key is 'Michigan', value is 'MI'
}

#the abbreviation on the left is the 'key' and the city's name on the right is the 'value' of the dictionary
cities = {
    "CA": "San Francisco", #the key is 'CA', value is 'San Francisco'
    "MI": "Detroit", #the key is 'MI', value is 'Detroit'
    "FL": "Jacksonville", #the key is 'FL', value is 'Jacksonville'
    "NY": "New York", #the key is 'NY', value is 'New York'
    "OR": "Portland" #the key is 'OR', value is 'Portland'
}
```